

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

**«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ
УНИВЕРСИТЕТ им. В. Г. Шухова»
(БГТУ им. В. Г. Шухова)**



Кафедра программного обеспечения вычислительной
техники и автоматизированных систем

Лабораторная работа №2

по дисциплине: «Параллельное программирование»
на тему: «Реализация параллелизма в рамках стандарта OpenMP.»

Выполнил: ст. группы ПВ-223
Дмитриев Андрей Александрович

Проверили:
доц. Островский Алексей Мичеславович,

Белгород, 2025

Цель работы: Изучить возможности стандарта OpenMP для создания многопоточных программ, изучить механизмы управления потоками и различные стратегии распределения работы между потоками, их влияние на производительность вычислений.

Цель работы обуславливает постановку и решение следующих **задач**:

1. Ознакомиться с основами OpenMP, включая компиляцию и запуск многопоточных программ.
2. Изучить работу командной модели потоков OpenMP.
3. Рассмотреть основы параллельного вычисления в цикле с разными стратегиями распределения работы между потоками (static, dynamic, guided, auto).
4. Выполнить индивидуальное задание, связанное с использованием стандарта OpenMP для реализации вычислительной задачи. Следует декомпозировать вычислительную задачу, вычленив сущности для потоковой обработки. Внимание! В процессе декомпозиции вычленяйте сущности различным образом, с тем чтобы рассмотреть стратегии распределения работ между потоками в условиях пропорциональных и диспропорциональных вычислительных нагрузок.
5. Провести анализ эффективности каждой стратегии, применительно к поставленной задаче, измеряя ускорение и масштабируемость решения при увеличении числа потоков. Описать наблюдения, сделав выводы о влиянии различных стратегий балансировки нагрузки на производительность вычислений.

Условие индивидуального задания:

Произвести расчёт следующего выражения:

$$S = \sum_{i=1}^N \frac{\sin(i) + \cos(2i) + i^3}{\sqrt{i+1} + \ln(i+1)}$$

Ход выполнения работы

Текст программы на языке C.

test.c

```
#include <omp.h>
#include <math.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>

uint64_t calc(size_t i)
{
    double dividend = sin(i) + cos(2 * i) + pow(i, 3);
    double divisor = sqrt(i + 1) + log(i + 1);
    return (uint64_t)(dividend / divisor);
}

uint64_t func(size_t num_iter, char key)
{
    uint64_t sum = 0;
    if (key == 's')
    {
        #pragma omp parallel for schedule(static) reduction(+ : sum)
        for (size_t i = 0; i < num_iter; i++)
        {
            sum += calc(i);
        }
    }
    else if (key == 'd')
    {
        #pragma omp parallel for schedule(dynamic) reduction(+ : sum)
        for (size_t i = 0; i < num_iter; i++)
        {
            sum += calc(i);
        }
    }
    else if (key == 'g')
    {
        #pragma omp parallel for schedule(guided) reduction(+ : sum)
        for (size_t i = 0; i < num_iter; i++)
        {
            sum += calc(i);
        }
    }
    else if (key == 'a')
    {
        #pragma omp parallel for schedule(auto) reduction(+ : sum)
        for (size_t i = 0; i < num_iter; i++)
        {
            sum += calc(i);
        }
    }
}
```

```

    }
    else
    {
        printf("Unknown schedule type: %c\n", key);
    }

    return sum;
}

int main(int argc, char const *argv[])
{
    if (argc != 3)
        return 1;

    size_t num_iter = atoll(argv[1]);
    char key = *argv[2];

    func(num_iter, key);

    return 0;
}

```

+вспомогательные скрипты, программы из предыдущей лабораторной работы

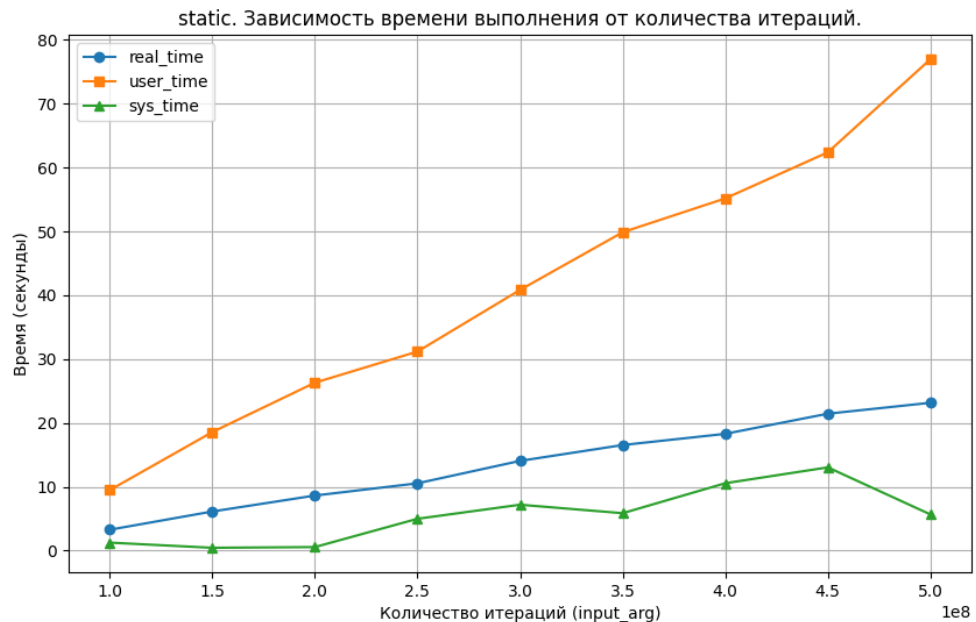
Сравнение опций OpenMP

Протоколы, логи, скриншоты, графики.

timetest_static.csv

```
sep=;
input_arg;real_time;user_time;sys_time
100000000;3.267;9.465;1.251
150000000;6.127;18.565;.442
200000000;8.625;26.301;.556
250000000;10.541;31.176;4.986
300000000;14.055;40.836;7.188
350000000;16.537;49.866;5.859
400000000;18.290;55.166;10.553
450000000;21.445;62.391;13.038
500000000;23.155;76.973;5.666
```

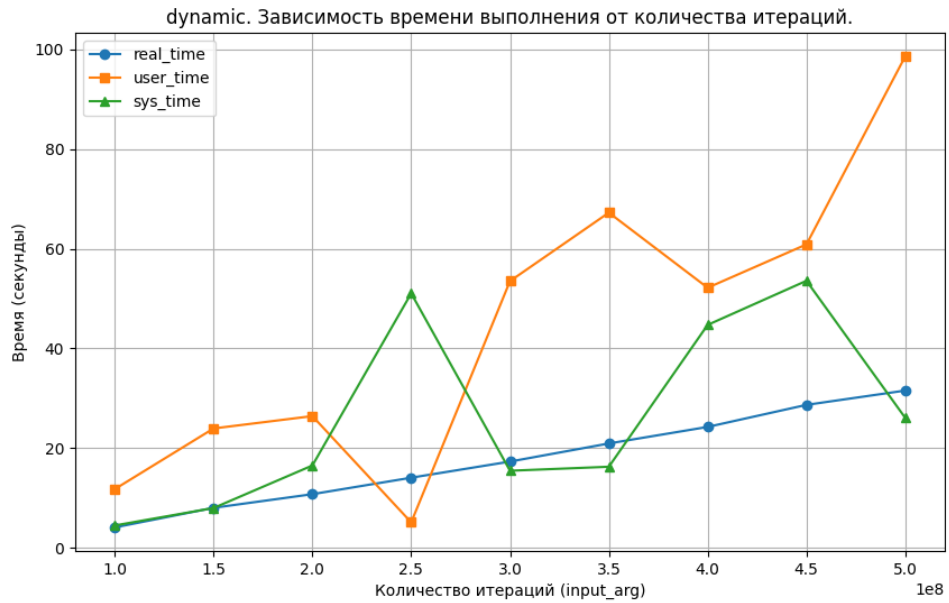
график:



timetest_dynamic.csv

```
sep=;
input_arg;real_time;user_time;sys_time
100000000;4.088;11.769;4.478
150000000;8.018;23.939;7.981
200000000;10.758;26.417;16.512
250000000;14.061;5.103;51.033
300000000;17.296;53.500;15.456
350000000;20.926;67.230;16.256
400000000;24.247;52.162;44.711
450000000;28.668;60.863;53.559
500000000;31.542;98.588;26.056
```

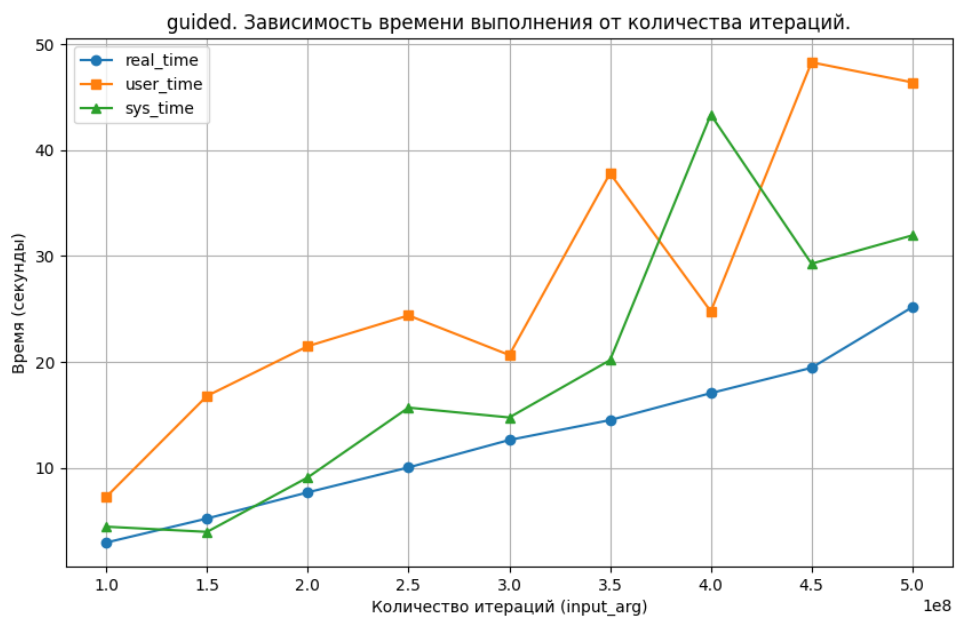
график:



timetest_guided.csv

```
sep=;
input_arg;real_time;user_time;sys_time
100000000;2.937;7.254;4.435
150000000;5.207;16.789;3.942
200000000;7.677;21.486;9.091
250000000;10.031;24.393;15.682
300000000;12.624;20.650;14.751
350000000;14.503;37.815;20.169
400000000;17.052;24.770;43.351
450000000;19.444;48.301;29.276
500000000;25.173;46.408;31.953
```

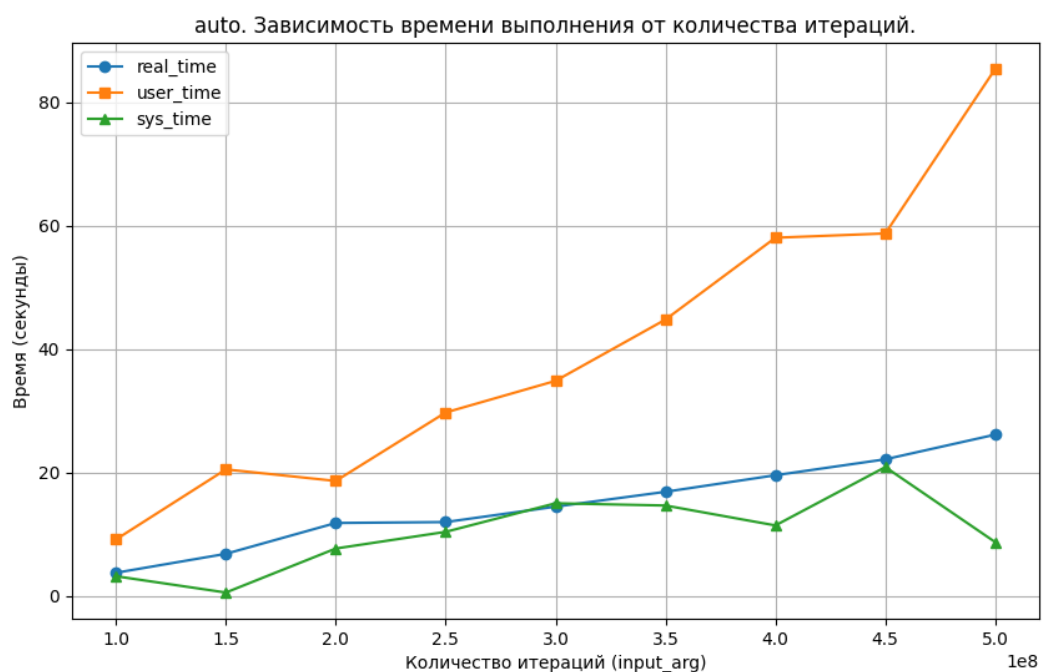
график:



timetest_auto.csv

```
sep=;
input_arg;real_time;user_time;sys_time
100000000;3.738;9.143;3.179
150000000;6.812;20.504;.533
200000000;11.815;18.633;7.673
250000000;11.964;29.739;10.378
300000000;14.465;34.867;15.014
350000000;16.876;44.813;14.647
400000000;19.561;58.081;11.418
450000000;22.144;58.773;20.884
500000000;26.146;85.459;8.710
```

график:



Замеры.

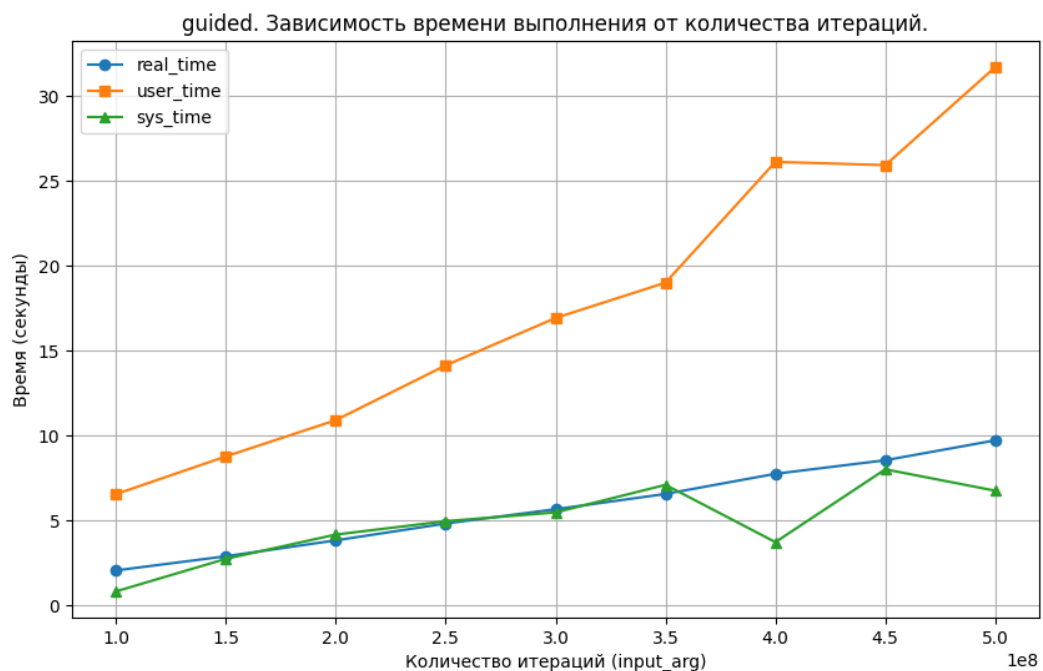
Сравнивая графики можно заметить, что в данном замере лидером оказалась опция `guided`, немного обогнав `static`. Следует предположение, что за счёт балансировки нагрузки получается выигрыш во времени. `Static` оказался также весьма быстрым за счёт однообразности вычислений. `Dynamic` имеет высокий расход на переключение потоков, поэтому проиграл по времени выполнения.

Ускоренный алгоритм без погрешности

Единственное существенное ускорение без увеличения погрешности, которое удалось обнаружить это замена `row(i, 3)` на `i*i*i`.

Изменения в коде:

```
uint64_t calc(size_t i)
{
    double dividend = sin(i) + cos(2 * i) + (i * i * i);
    double divisor = sqrt(i + 1) + log(i + 1);
    return (uint64_t)(dividend / divisor);
}
```

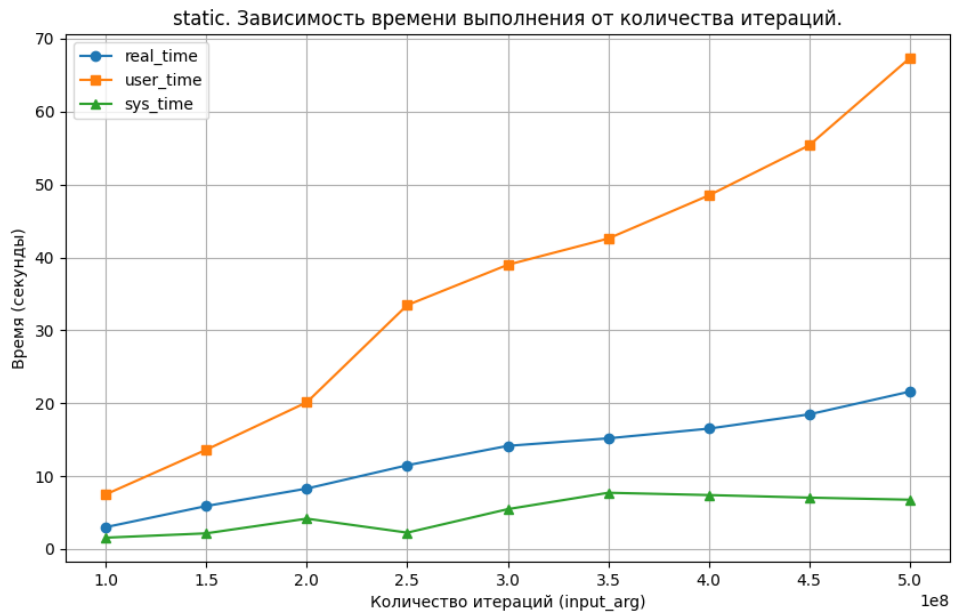


Протоколы, логи, скриншоты, графики.

timetest_static.csv

```
sep=;
input_arg;real_time;user_time;sys_time
100000000;3.000;7.497;1.557
150000000;5.900;13.625;2.152
200000000;8.294;20.126;4.168
250000000;11.491;33.458;2.246
300000000;14.149;38.988;5.470
350000000;15.182;42.588;7.717
400000000;16.515;48.496;7.402
450000000;18.471;55.376;7.049
500000000;21.590;67.326;6.766
```

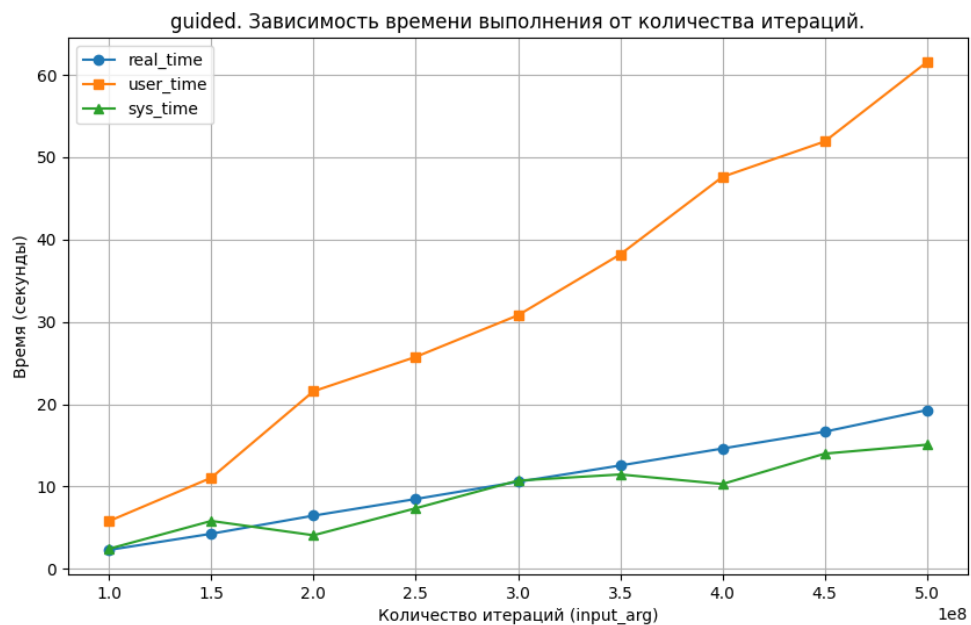
график:



timetest_guided.csv

```
sep=;
input_arg;real_time;user_time;sys_time
100000000;2.292;5.797;2.430
150000000;4.259;11.060;5.813
200000000;6.460;21.569;4.074
250000000;8.479;25.742;7.356
300000000;10.565;30.799;10.715
350000000;12.550;38.196;11.460
400000000;14.612;47.610;10.299
450000000;16.658;51.911;13.990
500000000;19.285;61.560;15.077
```

график:



Замеры.

Определённо данное ускорение заметно уменьшило время вычислений.

Ускоренный алгоритм с погрешностью

Помимо предыдущего ускорения можно заранее рассчитать синус и косинус выражения и составить их таблицы. Результат этих функций незначительно влияет на целый результат выражения, принимая значения от -1 до 1 и суммируясь с i^3 .

Изменения в коде:

```
#define PI 3.14159265358979323846
#define STEP 0.01
#define TABLE_SIZE ((int)(2 * PI / STEP) + 1)

double sin_table[TABLE_SIZE];
double cos_table[TABLE_SIZE];

void initializeTrigTables() {
    #pragma omp parallel for
    for (int i = 0; i < TABLE_SIZE; ++i) {
        double angle = i * STEP;
        sin_table[i] = sin(angle);
        cos_table[i] = cos(angle);
    }
}

double fastSin(double x) {
    x = fmod(x, 2 * PI);
    if (x < 0) x += 2 * PI;

    int index = (int)(x / STEP);
    return sin_table[index];
}

double fastCos(double x) {
    x = fmod(x, 2 * PI);
    if (x < 0) x += 2 * PI;

    int index = (int)(x / STEP);
    return cos_table[index];
}

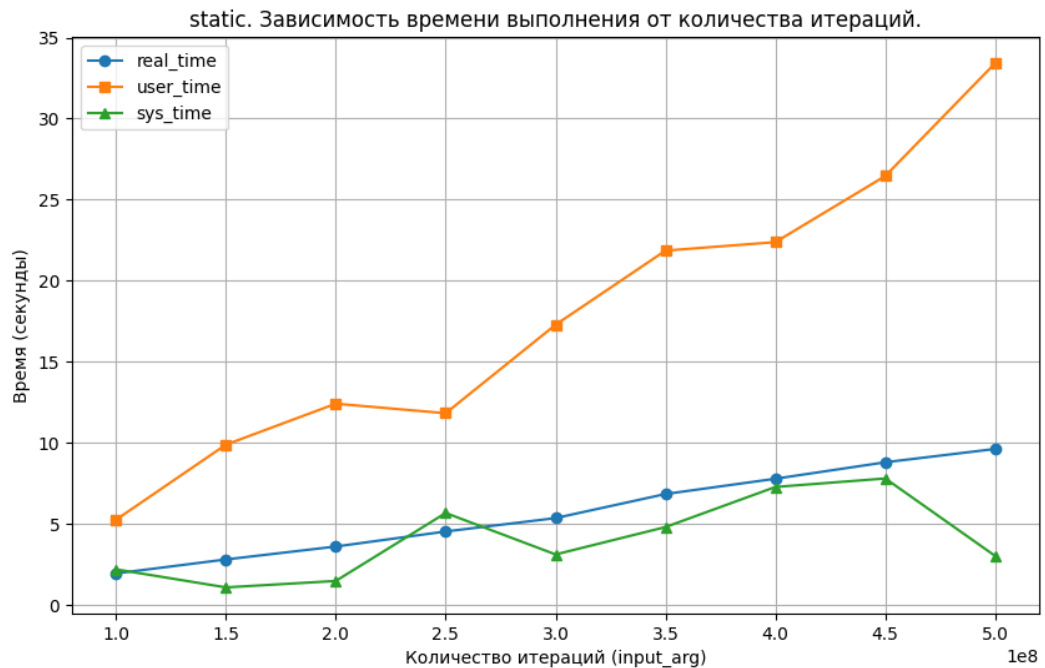
uint64_t calc(size_t i) {
    double dividend = fastSin(i) + fastCos(2 * i) + (i * i * i);
    double divisor = sqrt(i + 1) + log(i + 1);
    return (uint64_t)(dividend / divisor);
}
```

Протоколы, логи, скриншоты, графики.

timetest_static.csv

```
sep=;
input_arg;real_time;user_time;sys_time
100000000;1.966;5.272;2.218
150000000;2.828;9.903;1.106
200000000;3.625;12.429;1.502
250000000;4.553;11.842;5.693
300000000;5.378;17.296;3.137
350000000;6.862;21.867;4.825
400000000;7.804;22.388;7.296
450000000;8.819;26.467;7.821
500000000;9.640;33.424;3.042
```

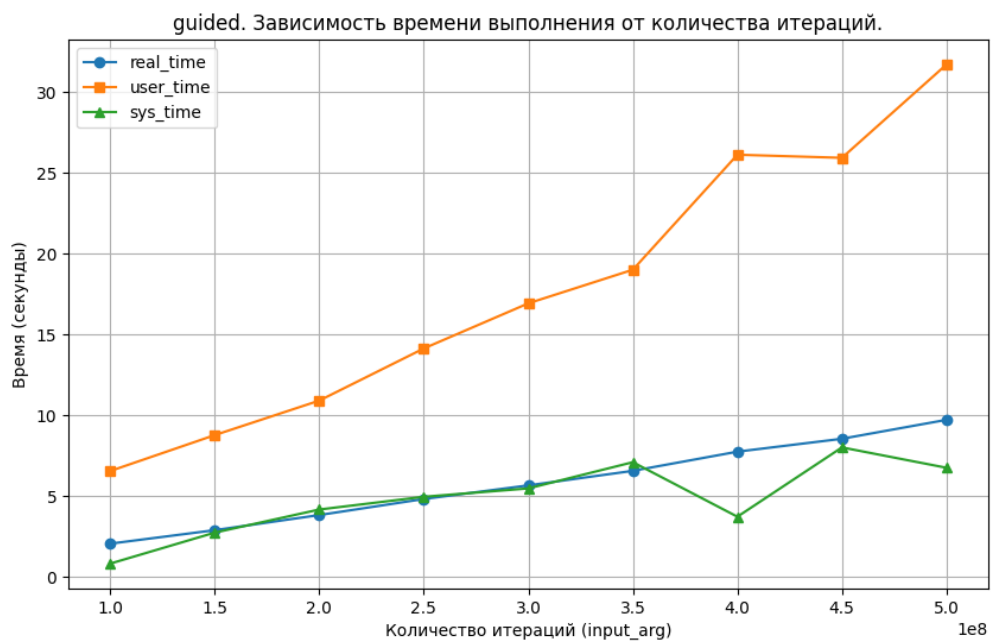
график:



timetest_guided.csv

```
sep=;
input_arg;real_time;user_time;sys_time
100000000;2.060;6.553;.826
150000000;2.895;8.775;2.743
200000000;3.830;10.910;4.172
250000000;4.823;14.136;4.960
300000000;5.665;16.927;5.478
350000000;6.565;19.007;7.100
400000000;7.751;26.120;3.727
450000000;8.549;25.928;8.014
500000000;9.721;31.701;6.755
```

график:



Замеры.

Данное ускорение дало гигантский прирост скорости (практически в 2 раза).
Но результат может быть с повышенной погрешностью.

Выводы

В ходе лабораторной работы было выполнено индивидуальное задание. Изучены возможности стандарта OpenMP для создания многопоточных программ, изучены механизмы управления потоками и различные стратегии распределения работы между потоками, их влияние на производительность вычислений.

Лучшая опция для расчёта подобных задач оказалась `guided`. В большем количестве случаев расчёт с ней происходил быстрее.

Данную задачу при применении оптимизаций удалось ускорить практически в 2 раза от первоначального параллельного алгоритма.